一句话读懂 Transformer

——以“我要吃苹果”的中译英为例，从原理到完整代码

目录

- 前言：为什么用一句话讲一个模型
- 第 1 章　翻译问题与 Transformer 的诞生
- 第 2 章　从文字到数字：分词、词表与嵌入
- 第 3 章　位置编码：让模型知道“谁在前谁在后”
- 第 4 章　注意力机制：Transformer 的心脏
- 第 5 章　多头注意力
- 第 6 章　前馈网络、残差连接与层归一化
- 第 7 章　编码器：把“我要吃苹果”读懂
- 第 8 章　解码器与掩码：一个词一个词地写出英文
- 第 9 章　训练：模型是怎么学会的
- 第 10 章　推理：逐步追踪“我要吃苹果”的翻译过程
- 第 11 章　完整代码
- 第 12 章　运行结果与解读
- 第 13 章　从玩具到真实系统
- 附录 A　张量形状速查表
- 附录 B　常见错误与排查
- 附录 C　练习题

前言：为什么用一句话讲一个模型

讲 Transformer 的材料很多，但大多数要么停在公式，要么直接丢出几百行工程代码，读者很难把“公式里的 Q、K、V”和“屏幕上翻译出来的那句英文”联系起来。

本书只做一件事：**跟着一句话走完全程**。

```
输入：我要吃苹果
输出：i want to eat an apple
```

从这句中文被切成词、变成数字、经过编码器、被解码器一个词一个词地“写”成英文——每一步都对应一个公式、一段代码、一个张量形状。读完你应该能：

1. 说清楚 Transformer 每个组件**为什么存在、解决什么问题**；
2. 手算一个最小的注意力例子；
3. 不依赖 `nn.Transformer`，从零写出一个能跑通的中译英模型，并看懂它的每一行。

前置知识：会 Python，知道矩阵乘法，大致了解神经网络“前向 → 损失 → 反向传播 → 更新参数”的流程。用过 PyTorch 更好，没用过也能跟上。

运行环境：Python 3.8+，PyTorch 1.12+。全部代码在普通笔记本 CPU 上约 20 秒训练完毕，不需要 GPU。

第 1 章　翻译问题与 Transformer 的诞生

1.1 机器翻译要解决什么

翻译是一个**序列到序列**（Sequence-to-Sequence, Seq2Seq）问题：输入一串符号，输出另一串符号，两者长度可以不同，顺序也未必一一对应。

中文	我	要	吃	苹果		
英文	i	want	to	eat	an	apple

注意几件事：

- 中文 4 个词，英文 6 个词——**长度不等**；
- “to” 和 “an” 在中文里**没有对应词**，是英语语法要求的；
- “an” 的选择取决于**后面**的 “apple” 以元音开头——生成 “an” 时模型必须已经“知道”接下来要写 apple；
- “他要吃苹果”要译成 “he **wants**”——动词形式取决于**前面**的主语。

所以一个好的翻译模型必须：**全局理解源句**，并在生成每个目标词时**有选择地关注源句的相关部分**。

1.2 RNN 时代的困境

2014–2017 年主流方案是 RNN（LSTM/GRU）编码器-解码器：

```
编码器：我 → 要 → 吃 → 苹果 → [一个固定向量 c]
解码器：c → i → want → to → eat → an → apple
```

它有两个根本问题：

1. **信息瓶颈**：整句话被压缩进一个固定长度向量 `c`，句子一长就丢信息。2015 年 Bahdanau 等人引入“注意力”缓解了这一点——解码时允许回看编码器的每个位置。
2. **无法并行**：RNN 第 t 步必须等第 t−1 步算完，GPU 的并行能力用不上，训练慢；且距离远的词之间信号要穿过很多步，容易梯度消失。

1.3 “Attention Is All You Need”

2017 年 Google 的论文 *Attention Is All You Need*（Vaswani et al.）提出了一个大胆的想法：**既然注意力这么好用，干脆把 RNN 整个扔掉，只用注意力**。

这就是 Transformer。它的核心收益：

- 任意两个词之间只隔**一步**注意力计算，长距离依赖不再困难；
- 一个句子的所有位置**同时计算**，训练高度并行；

- 结构规整（同一种层堆叠 N 次），易于扩展——这也是后来 BERT、GPT 乃至今天所有大语言模型的基础。

1.4 全局架构一览

```
【编码器】（同样结构的层堆叠 N 次）
  输入：我 要 吃 苹果
    ↓ 词嵌入 + 位置编码
    ↓ 多头自注意力        → 残差 + LayerNorm
    ↓ 前馈网络            → 残差 + LayerNorm
    ↓
  输出：memory（源句的上下文表示，送入解码器每一层的交叉注意力）

【解码器】（同样结构的层堆叠 N 次）
  输入：<bos> i want to eat an    （已生成的英文）
    ↓ 词嵌入 + 位置编码
    ↓ 带掩码的多头自注意力 → 残差 + LayerNorm
    ↓ 多头交叉注意力       → 残差 + LayerNorm   ← Q 来自解码器，K/V 来自 memory
    ↓ 前馈网络             → 残差 + LayerNorm
    ↓ 线性层 + Softmax
  输出：i want to eat an apple <eos>    （每个位置预测下一个词）
```

后面每一章拆解图中的一个环节。

第 2 章　从文字到数字：分词、词表与嵌入

神经网络只能处理数字。第一步是把“我要吃苹果”变成一串向量。

2.1 分词（Tokenization）

先把句子切成**token**（模型处理的最小单位）。本书为了聚焦模型本身，采用最简单的方式：

- 中文：按**词**切，用空格分隔 → `我 要 吃 苹果`
- 英文：按空格切 → `i want to eat an apple`

> 真实系统用 BPE / SentencePiece 等**子词**算法，能处理没见过的词（见第 13 章）。原理上无区别：都是“文本 → token 序列”。

2.2 词表（Vocabulary）与特殊符号

给每个 token 分配一个整数 id。另外加四个特殊符号：

符号	作用
`<pad>`	填充。一个批次里句子长短不一，短的用它补齐
`<bos>`	Begin of Sentence，告诉解码器“开始写”
`<eos>`	End of Sentence，模型输出它表示“写完了”
`<unk>`	词表外的词

本书语料构建出的中文词表（17 个）前几项：

```
0:<pad> 1:<bos> 2:<eos> 3:<unk> 4:我 5:要 6:吃 7:苹果 ...
```

于是 “我要吃苹果” 编码为：

```
<bos> 我 要 吃 苹果 <eos>
  1    4  5  6   7    2
```

对应代码（第 11 章 `Vocab.encode`）：

```
        ids = [self.stoi.get(t, self.stoi[UNK]) for t in sentence.split()]
        ids = [self.stoi[BOS]] + ids + [self.stoi[EOS]]
```

2.3 词嵌入（Embedding）

整数 id 本身没有“含义”——7 并不比 6 更像水果。所以把每个 id 映射成一个 **d_model 维的稠密向量**（本书 d_model = 128）：

$$\text{Embedding}: \{0, 1, \ldots, |V|-1\} \to \mathbb{R}^{d_{model}}$$

实现上就是一张 `|V| × d_model` 的查找表，每一行是一个可学习的向量。训练结束后，“苹果”和“香蕉”的向量会比较接近，因为它们出现在相似的上下文里。

```
class TokenEmbedding(nn.Module):
    def forward(self, x):                # x: (B, L) 整数
        return self.emb(x) * math.sqrt(self.d_model)   # → (B, L, d_model)
```

为什么乘 $\sqrt{d_{model}}$？ 嵌入向量初始化时数值较小，而接下来要加上的位置编码取值在 [−1, 1]。乘以 $\sqrt{128} \approx 11.3$ 让词义信息不被位置信息淹没。这是原论文的做法。

此时“我要吃苹果”变成了形状为 `(1, 6, 128)` 的张量：1 个句子，6 个 token，每个 128 维。

第 3 章　位置编码：让模型知道“谁在前谁在后”

3.1 问题：注意力是“顺序盲”的

下一章会看到，注意力本质上是对一组向量求**加权和**。加权和不关心顺序——把“我要吃苹果”打乱成“苹果吃要我”，每个词得到的注意力结果完全一样（只是位置跟着换了）。

但“我吃苹果”和“苹果吃我”意思天差地别。所以必须**显式地把位置信息注入**每个词的向量。

3.2 正弦位置编码

原论文的方案：对位置 `pos`、维度 `i`：

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

然后直接**加**到词嵌入上：`x = Embedding(token) + PE(pos)`。

用 d_model = 4 的迷你例子看具体数值：

位置	token	dim0 (sin, 高频)	dim1 (cos, 高频)	dim2 (sin, 低频)	dim3 (cos, 低频)
0	`<bos>`	0.00	1.00	0.00	1.00
1	我	0.84	0.54	0.01	1.00
2	要	0.91	−0.42	0.02	1.00
3	吃	0.14	−0.99	0.03	1.00

可以把它想象成一组**转速不同的时钟指针**：低维度转得快（区分相邻位置），高维度转得慢（区分远距离位置）。每个位置都得到一个独一无二的“指纹”。

3.3 为什么选 sin/cos?

- **有界**：值域 [−1, 1]，不会随句子变长而爆炸；
- **相对位置可线性表示**：对任意固定偏移 k，PE_{pos+k} 是 PE_{pos} 的一个线性变换（旋转），模型容易学到“往后数 k 个词”这类模式；
- **无需学习**，可外推到训练中没见过的长度。

> 现代大模型多用 RoPE（旋转位置编码）或可学习位置向量，思想一脉相承。

3.4 代码

```
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(max_len).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)   # 偶数维
        pe[:, 1::2] = torch.cos(pos * div)   # 奇数维
```

`div` 用 `exp(-log(10000)·2i/d)` 计算 $1/10000^{2i/d}$，数值上更稳定。`register_buffer` 让它随模型移动到 GPU，但不参与训练。

第 4 章　注意力机制：Transformer 的心脏

4.1 直觉：一次“软查询”

翻译到 “apple” 时，模型应该主要看“苹果”；翻译到 “want” 时，主要看“要”。注意力就是让每个位置**自己决定该看谁、看多少**的机制。

借用数据库的比喻：

角色	含义	类比
Query（查询） Q	“我现在想找什么”	搜索框里的关键词
Key（键） K	“我这里有什么，可供匹配”	每篇文档的标题
Value（值） V	“匹配上之后，我提供的内容”	文档正文

普通数据库是**硬匹配**：命中就返回，否则不返回。注意力是**软匹配**：Query 和每个 Key 算一个相似度，归一化成权重，再按权重把所有 Value 加起来。

4.2 缩放点积注意力

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

分四步：

1. **打分** $QK^{\top}$：每个 Query 和每个 Key 做点积，越相似分数越高；
2. **缩放** $\div\sqrt{d_k}$：防止维度大时点积过大，导致 softmax 饱和、梯度接近 0；
3. **归一化** softmax：每一行变成和为 1 的概率分布——这就是“注意力权重”；
4. **加权求和** $\times V$：按权重混合所有 Value。

Q、K、V 从哪里来？都由输入 x 经过三个**可学习的线性变换**得到：

$$Q=XW^{Q},\quad K=XW^{K},\quad V=XW^{V}$$

同一个词，扮演三种角色时用三套不同的“面孔”。模型通过训练学会该用什么样的 Q 去找什么样的 K。

4.3 手算一个例子

取 “我 要 吃 苹果” 四个词，假设 d_k = 2，经过线性变换后（为了好算，令 Q = K = V）：

	向量
我	[1, 0]
要	[0, 1]
吃	[1, 1]
苹果	[0, 2]

第 1 步：打分并缩放 $QK^\top/\sqrt{2}$

Q ＼ K	我	要	吃	苹果
我	0.71	0.00	0.71	0.00
要	0.00	0.71	0.71	1.41
吃	0.71	0.71	1.41	1.41
苹果	0.00	1.41	1.41	2.83

例：吃·苹果 = $1\times0 + 1\times2 = 2$，再除以 $\sqrt{2} \approx 1.41$。

第 2 步：逐行 softmax，得到注意力权重：

Q ＼ K	我	要	吃	苹果
我	0.33	0.17	0.33	0.17
要	0.11	0.22	0.22	0.45
吃	0.17	0.17	0.33	0.33
苹果	0.04	0.16	0.16	**0.65**

读法：“吃”这一行表示它把 33% 的注意力放在自己、33% 放在“苹果”——动词在关注它的宾语。

第 3 步：加权求和 以“吃”为例：

$$0.17\cdot[1,0] + 0.17\cdot[0,1] + 0.33\cdot[1,1] + 0.33\cdot[0,2] \approx [0.50,\ 1.17]$$

新的“吃”向量融合了整句话的信息，尤其是“苹果”。这就是**上下文化表示**：同一个“吃”，在“吃苹果”和“吃亏”里会得到不同的向量。

完整输出：

	注意力输出
我	[0.67, 0.83]
要	[0.33, 1.34]
吃	[0.50, 1.17]
苹果	[0.20, 1.61]

4.4 掩码（Mask）

有时我们**不允许**某些位置被看到：

- **Padding 掩码**：`<pad>` 是填充物，不该被关注；
- **因果掩码**：解码器生成第 t 个词时，不能偷看第 t+1 个词（第 8 章详述）。

做法：在 softmax **之前**把不允许位置的分数设为 $-\infty$，softmax 后其权重恰好为 0。

```
        def scaled_dot_product_attention(q, k, v, mask=None):
            d_k = q.size(-1)
            scores = q @ k.transpose(-2, -1) / math.sqrt(d_k)
            if mask is not None:
                scores = scores.masked_fill(~mask, float("-inf"))   # mask 中 True=允许
            attn = torch.softmax(scores, dim=-1)
            return attn @ v, attn
```

4.5 自注意力 vs 交叉注意力

同一个公式，Q/K/V 来源不同就有两种用法：

类型	Q 来自	K、V 来自	用在哪	作用
自注意力	本序列	本序列	编码器、解码器	句子内部词与词互相理解
交叉注意力	解码器	**编码器输出**	解码器	生成英文时回看中文源句

第 5 章　多头注意力

5.1 一个头不够

一个注意力头只能产生一种权重分布。但语言中同时存在多种关系：

- “吃”↔“苹果”：动宾关系
- “我”↔“要”：主谓关系
- “要”↔“吃”：情态动词与主动词

多头注意力让模型并行地用 h 组不同的 W^Q, W^K, W^V，各自关注不同类型的关系，再把结果拼起来：

$$\text{MultiHead}\left(Q, K, V\right) = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_h\right) W^O$$

$$\text{head}_i = \text{Attention}\left(QW_i^Q,\ KW_i^K,\ VW_i^V\right)$$

5.2 维度如何拆分

本书 d_model = 128，h = 4，所以每个头 d_k = 128 / 4 = 32。总计算量和单个 128 维的头差不多，但获得了 4 个“视角”。

实现上不必真的建 4 个小线性层——用一个 128→128 的大线性层，再 reshape 成 4 份即可：

```
(B, L, 128)  --view-->  (B, L, 4, 32)  --transpose-->  (B, 4, L, 32)
```

这样 4 个头的注意力可以在一次批量矩阵乘法里同时算完。

```
        def _split(self, x):                        # (B,L,d) -> (B,h,L,d_k)
            B, L, _ = x.shape
            return x.view(B, L, self.h, self.d_k).transpose(1, 2)

        def forward(self, query, key, value, mask=None):
            q = self._split(self.w_q(query))
            k = self._split(self.w_k(key))
            v = self._split(self.w_v(value))
            out, self.attn = scaled_dot_product_attention(q, k, v, mask)
            B, _, L, _ = out.shape
            out = out.transpose(1, 2).contiguous().view(B, L, self.h * self.d_k)   # 拼接
            return self.dropout(self.w_o(out))                                      # W^O 融合
```

注意 `forward` 的参数是 `query, key, value` 三个——自注意力时传 `(x, x, x)`，交叉注意力时传 `(y, memory, memory)`。同一个类，两种用法。

第 6 章　前馈网络、残差连接与层归一化

注意力负责“词与词之间交换信息”，但每个词拿到信息后还需要**自己消化**。这一章的三个组件就是干这个的，同时保证深层网络能训得动。

6.1 逐位置前馈网络（FFN）

$$\mathrm{FFN}(x) = W_2\,\mathrm{ReLU}(W_1 x + b_1) + b_2$$

- 先升维到 d_ff（本书 256，原论文 2048 = 4×512），再降回 d_model；
- **对每个位置独立应用同一个网络**——位置之间不交互（交互是注意力的事）；
- 研究表明 FFN 层承担了大量“知识存储”的功能。

6.2 残差连接（Residual Connection）

$$\mathrm{output} = x + \mathrm{Sublayer}(x)$$

把子层的输入直接加到输出上。好处：

- 梯度可以通过“+”这条捷径直接回传，深层网络不易梯度消失；
- 子层只需要学“在 x 基础上改多少”（残差），而不是从头生成，学起来更容易。

6.3 层归一化（Layer Normalization）

对每个 token 的 d_model 维向量做归一化（均值 0、方差 1），再乘以可学习的缩放 γ、加偏移 β：

$$\mathrm{LN}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

作用：稳定每层输入的数值分布，让训练更平稳。为什么不用 BatchNorm？因为句子长度不一、批次常常很小，按 token 自身的特征维度归一化更可靠。

6.4 组合起来：Add & Norm

原论文（Post-LN）的写法：

$$x = \mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$$

> 现在的大模型多用 **Pre-LN**：$x = x + \mathrm{Sublayer}(\mathrm{LayerNorm}(x))$，深层时更稳定。本书遵循原论文，第 13 章会再提到。

第 7 章　编码器：把“我要吃苹果”读懂

7.1 一个编码器层

```
x  ──►  多头自注意力  ──►  与 x 相加（残差）  ──►  LayerNorm  ──►  x'
x' ──►  前馈网络 FFN  ──►  与 x' 相加（残差）  ──►  LayerNorm  ──►  输出
```

```python
class EncoderLayer(nn.Module):
    def forward(self, x, src_mask):
        x = self.ln1(x + self.self_attn(x, x, x, src_mask))
        x = self.ln2(x + self.ffn(x))
        return x
```

只有两行，却包含了前三章的全部内容。

7.2 堆叠 N 层

把编码器层堆叠 N 次（原论文 N=6，本书 N=2）。每过一层，每个词的表示就融合了更多、更抽象的上下文信息：

- 第 1 层：“吃”知道了旁边有“苹果”；
- 第 2 层：“吃”知道了“我**要**吃苹果”是一个意愿表达。

7.3 编码器的输出：memory

“我要吃苹果”（加上 `<bos>`、`<eos>` 共 6 个 token）经过编码器后，得到形状 `(1, 6, 128)` 的张量，称为 **memory**。它是解码器在整个生成过程中反复查阅的“源句理解”。

关键点：**编码器只需运行一次**。之后解码器每生成一个英文词，都只是重新查阅这份 memory。

7.4 源端掩码

批量训练时句子会被 `<pad>` 补齐，编码器不能关注这些位置：

```python
@staticmethod
def make_src_mask(src):                       # (B, Ls) → (B, 1, 1, Ls)
    return (src != PAD_IDX).unsqueeze(1).unsqueeze(2)
```

形状 `(B, 1, 1, Ls)` 可以广播到注意力分数的 `(B, h, Lq, Ls)`：对所有头、所有查询位置，统一屏蔽 pad 列。

第 8 章　解码器与掩码：一个词一个词地写出英文

8.1 自回归生成

解码器**一次生成一个词**，并把已生成的词作为下一步的输入：

```
输入 <bos>                      → 输出 i
输入 <bos> i                    → 输出 want
输入 <bos> i want               → 输出 to
...
输入 <bos> i want to eat an apple → 输出 <eos>    (结束)
```

这叫**自回归**（autoregressive）：每一步的输出依赖之前所有步的输出。

8.2 解码器层：三个子层

```
y ──► 带因果掩码的自注意力 ──► Add&Norm ──► 交叉注意力(看 memory) ──► Add&Norm ──► FFN ──► Add&Norm
```

子层	Q	K, V	回答的问题
掩码自注意力	已生成的英文	已生成的英文	“我已经写了什么？语法上接下来该是什么？”
交叉注意力	已生成的英文	**中文 memory**	“源句中哪部分还没翻译？现在该翻哪个词？”
FFN	—	—	消化整合

```
        class DecoderLayer(nn.Module):
            def forward(self, y, memory, tgt_mask, src_mask):
                y = self.ln1(y + self.self_attn(y, y, y, tgt_mask))
                y = self.ln2(y + self.cross_attn(y, memory, memory, src_mask))
                y = self.ln3(y + self.ffn(y))
                return y
```

8.3 因果掩码：不许偷看答案

训练时我们把整句正确英文一次性喂给解码器（为了并行，见第 9 章）。但如果不加限制，模型在预测“want”时就能直接看到后面的“want”——等于抄答案，学不到东西。

因果掩码（causal mask，也叫 look-ahead mask）是一个下三角矩阵，保证位置 t 只能看到位置 ⩽ t：

```
          <bos>  i   want  to   eat   an
<bos>   [   1    0    0    0    0     0  ]
i       [   1    1    0    0    0     0  ]
want    [   1    1    1    0    0     0  ]
to      [   1    1    1    1    0     0  ]
```

```
eat     [  1    1    1    1    1    0  ]
an      [  1    1    1    1    1    1  ]
```

1 = 可以看，0 = 被屏蔽（分数设为 −∞）。再与 padding 掩码取“与”：

```
        @staticmethod
        def make_tgt_mask(tgt):                          # (B, Lt) → (B, 1, Lt, Lt)
            Lt = tgt.size(1)
            pad = (tgt != PAD_IDX).unsqueeze(1).unsqueeze(2)
            causal = torch.tril(torch.ones(Lt, Lt, dtype=torch.bool, device=tgt.device))
            return pad & causal
```

8.4 输出层：从向量到单词

解码器最后一层输出每个位置的 128 维向量，经过一个线性层投影到英文词表大小（27），再 softmax 得到“下一个词是谁”的概率分布：

```
        self.generator = nn.Linear(d_model, tgt_vocab)   # 128 → 27
```

第 9 章　训练：模型是怎么学会的

9.1 训练数据

本书构造了一个 22 句的迷你平行语料，覆盖“主语 × 要/不要/喜欢 × 吃/喝 × 食物”的组合：

```
        ("我 要 吃 苹果",   "i want to eat an apple"),
        ("他 要 吃 香蕉",   "he wants to eat a banana"),
        ("我 不 要 喝 水",  "i do not want to drink water"),
        ("她 喜欢 水",      "she likes water"),
        ...
```

刻意设计的语言现象：

- **主谓一致**：我/你 → want，他/她 → wants
- **冠词**：苹果 → an apple，香蕉 → a banana，水/牛奶 → 无冠词
- **否定**：不要 → do not want
- **一对多**：一个“要”对应 “want to” 两个词

为了检验模型是否真的**学到了规律**而不是死记硬背，“我不要喝牛奶”、“她要吃苹果”等测试句中，有的组合不在训练集里。

9.2 Teacher Forcing：输入右移一位

训练时解码器的输入和目标错开一位：

```
目标句 token:    <bos>  i   want  to  eat  an  apple  <eos>

解码器输入:      <bos>  i   want  to  eat  an  apple          (去掉最后一个)
预测目标:         i    want  to   eat  an  apple <eos>        (去掉第一个)
```

```
        tgt_in, tgt_out = tgt[:, :-1], tgt[:, 1:]
```

每个位置都喂入**正确**的历史，让模型预测下一个词。结合因果掩码，**一次前向就能同时训练所有 7 个位置**——这正是 Transformer 比 RNN 训练快的关键。

9.3 损失函数：交叉熵 + 标签平滑

每个位置是一个 27 分类问题，用交叉熵损失：

$$\mathcal{L} = -\sum_{t} \log P\left(y_t^* \mid y_{<t}, x\right)$$

```
        loss_fn = nn.CrossEntropyLoss(ignore_index=PAD_IDX, label_smoothing=0.1)
```

- `ignore_index=PAD_IDX`：pad 位置不计入损失；

- `label_smoothing=0.1`：把正确答案的目标概率从 1.0 降到约 0.9，剩下的均分给其他词。防止模型过度自信，提高泛化。原论文同样使用 0.1。

> 因为标签平滑，损失**不会降到 0**。本书训练最终 loss 约 0.63，这是正常的——它的理论下限大约就在这个位置。

9.4 优化器

```
        opt = torch.optim.Adam(model.parameters(), lr=5e-4, betas=(0.9, 0.98), eps=1e-9)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

- Adam 的 β 参数取自原论文；
- 梯度裁剪防止偶发的大梯度破坏训练；
- 原论文还使用了 **warmup 学习率调度**（先线性升高再按 $step^{-0.5}$ 衰减）。在本书的小模型、小数据上用固定学习率就足够，大规模训练时 warmup 几乎是必需的。

9.5 一个训练步的完整流程

```
make_batch       → src (B, Ls), tgt (B, Lt)
切分             → tgt_in (B, Lt-1), tgt_out (B, Lt-1)
model(src,tgt_in)→ logits (B, Lt-1, 27)
展平             → logits (B·(Lt-1), 27), tgt_out (B·(Lt-1),)
CrossEntropy     → 标量 loss
backward + step  → 更新 671,643 个参数
```

第 10 章　推理：逐步追踪“我要吃苹果”的翻译过程

训练完毕，模型参数固定。现在真正翻译“我要吃苹果”。

10.1 推理与训练的区别

	训练	推理
解码器输入	正确答案（teacher forcing）	**模型自己上一步的输出**
并行性	所有位置一次算完	必须一步一步来
Dropout	开启	关闭（`model.eval()`）

10.2 贪心解码

每一步取概率最大的词：

```
memory = model.encode(src, src_mask)                # 编码器只跑一次
ys = [[<bos>]]
循环:
    out = model.decode(ys, memory, ...)             # 解码已生成的序列
    next_id = argmax(softmax(generator(out[:, -1]))) # 只看最后一个位置
    ys = ys + [next_id]
    if next_id == <eos>: 停止
```

10.3 实际运行轨迹

以下是本书代码真实运行的输出（第 12 章有完整日志）：

步骤	解码器已有输入	预测下一个词	概率
1	`<bos>`	**i**	0.931
2	`<bos> i`	**want**	0.915
3	`<bos> i want`	**to**	0.917
4	`<bos> i want to`	**eat**	0.916
5	`<bos> i want to eat`	**an**	0.925
6	`<bos> i want to eat an`	**apple**	0.933
7	`<bos> i want to eat an apple`	**`<eos>`**	0.924

最终结果：

```
我要吃苹果  ->  i want to eat an apple
```

> 概率都在 0.92 左右而不是接近 1.0，同样是标签平滑的效果——模型被训练成“有把握但不过度自信”。

10.4 解码策略的其他选择

- **Beam Search**：每步保留概率最高的 k 条候选序列，最后选整体概率最高的。翻译质量通常比贪心好，是机器翻译的标准做法；
- **采样**（Top-k / Top-p / Temperature）：按概率随机抽取，多用于需要多样性的文本生成（如聊天、写作）。

第 11 章　完整代码

以下代码保存为 `transformer_zh2en.py`，直接运行 `python transformer_zh2en.py` 即可。所有组件从零实现，**没有**使用 `nn.Transformer` 或任何第三方 NLP 库。

结构：

部分	内容	对应章节
第 1 部分	数据、词表、批处理	第 2 章、第 9 章
第 2 部分	嵌入、位置编码、注意力、多头、FFN、编码器/解码器层、整体模型、掩码	第 2–8 章
第 3 部分	训练循环	第 9 章
第 4 部分	贪心解码、交叉注意力可视化	第 10、12 章

```
# -*- coding: utf-8 -*-
"""
从零实现 Transformer（Vaswani et al., 2017），完成中译英：
    我要吃苹果  ->  i want to eat an apple
依赖：Python 3.8+，PyTorch 1.12+。CPU 上约 1 分钟训练完毕。
运行：python transformer_zh2en.py
"""
import math
import random
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(42)
random.seed(42)
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# ============================================================
# 第 1 部分：数据 —— 一个迷你中英平行语料
# ============================================================
# 中文按“词”切分（用空格分隔，等价于做好了分词），英文按空格切分。
PAIRS = [
    ("我 要 吃 苹果",   "i want to eat an apple"),
    ("我 要 吃 香蕉",   "i want to eat a banana"),
    ("我 要 喝 水",     "i want to drink water"),
    ("我 要 喝 牛奶",   "i want to drink milk"),
    ("你 要 吃 苹果",   "you want to eat an apple"),
    ("你 要 喝 水",     "you want to drink water"),
    ("他 要 吃 香蕉",   "he wants to eat a banana"),
    ("他 要 喝 牛奶",   "he wants to drink milk"),
    ("她 要 吃 苹果",   "she wants to eat an apple"),
    ("她 要 喝 水",     "she wants to drink water"),
    ("我 喜欢 苹果",    "i like apples"),
    ("我 喜欢 香蕉",    "i like bananas"),
    ("你 喜欢 牛奶",    "you like milk"),
    ("他 喜欢 苹果",    "he likes apples"),
    ("她 喜欢 水",      "she likes water"),
```

```
    ("我 吃 苹果",      "i eat an apple"),
    ("你 吃 香蕉",      "you eat a banana"),
    ("他 吃 苹果",      "he eats an apple"),
    ("她 喝 牛奶",      "she drinks milk"),
    ("我 不 要 吃 苹果", "i do not want to eat an apple"),
    ("我 不 要 喝 水",  "i do not want to drink water"),
    ("你 不 要 吃 香蕉", "you do not want to eat a banana"),
]

PAD, BOS, EOS, UNK = "<pad>", "<bos>", "<eos>", "<unk>"
SPECIALS = [PAD, BOS, EOS, UNK]


class Vocab:
    """词表: token <-> id 的双向映射"""

    def __init__(self, sentences):
        self.itos = list(SPECIALS)
        for s in sentences:
            for tok in s.split():
                if tok not in self.itos:
                    self.itos.append(tok)
        self.stoi = {t: i for i, t in enumerate(self.itos)}

    def __len__(self):
        return len(self.itos)

    def encode(self, sentence, add_bos_eos=True):
        ids = [self.stoi.get(t, self.stoi[UNK]) for t in sentence.split()]
        if add_bos_eos:
            ids = [self.stoi[BOS]] + ids + [self.stoi[EOS]]
        return ids

    def decode(self, ids):
        toks = []
        for i in ids:
            t = self.itos[i]
            if t == EOS:
                break
            if t not in (PAD, BOS):
                toks.append(t)
        return " ".join(toks)


SRC_VOCAB = Vocab([p[0] for p in PAIRS])
TGT_VOCAB = Vocab([p[1] for p in PAIRS])
PAD_IDX = 0  # 两个词表中 <pad> 都是 0


def make_batch(pairs):
    """把若干句子对补齐成同长度张量"""
    src = [SRC_VOCAB.encode(s) for s, _ in pairs]
    tgt = [TGT_VOCAB.encode(t) for _, t in pairs]
    max_s, max_t = max(map(len, src)), max(map(len, tgt))
    src = [x + [PAD_IDX] * (max_s - len(x)) for x in src]
    tgt = [x + [PAD_IDX] * (max_t - len(x)) for x in tgt]
    return torch.tensor(src, device=DEVICE), torch.tensor(tgt, device=DEVICE)


# ============================================================
# 第 2 部分: 模型组件
```

```python
# ============================================================
class TokenEmbedding(nn.Module):
    """词嵌入，并按论文乘以 sqrt(d_model)"""

    def __init__(self, vocab_size, d_model):
        super().__init__()
        self.emb = nn.Embedding(vocab_size, d_model, padding_idx=PAD_IDX)
        self.d_model = d_model

    def forward(self, x):                       # x: (B, L)
        return self.emb(x) * math.sqrt(self.d_model)


class PositionalEncoding(nn.Module):
    """正弦位置编码：PE(pos,2i)=sin(pos/10000^(2i/d)), PE(pos,2i+1)=cos(...)"""

    def __init__(self, d_model, max_len=512, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(max_len).unsqueeze(1).float()           # (max_len, 1)
        div = torch.exp(torch.arange(0, d_model, 2).float()
                        * (-math.log(10000.0) / d_model))           # (d_model/2,)
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer("pe", pe.unsqueeze(0))                # (1, max_len, d)

    def forward(self, x):                       # x: (B, L, d)
        return self.dropout(x + self.pe[:, : x.size(1)])


def scaled_dot_product_attention(q, k, v, mask=None):
    """Attention(Q,K,V) = softmax(QK^T / sqrt(d_k)) V
    q,k,v: (B, h, L, d_k); mask: 可广播到 (B, h, Lq, Lk)，True 表示“允许看”
    """
    d_k = q.size(-1)
    scores = q @ k.transpose(-2, -1) / math.sqrt(d_k)              # (B,h,Lq,Lk)
    if mask is not None:
        scores = scores.masked_fill(~mask, float("-inf"))
    attn = torch.softmax(scores, dim=-1)
    return attn @ v, attn


class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.d_k = n_heads, d_model // n_heads
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_o = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)
        self.attn = None  # 保存最近一次注意力权重，便于可视化

    def _split(self, x):                        # (B,L,d) -> (B,h,L,d_k)
        B, L, _ = x.shape
        return x.view(B, L, self.h, self.d_k).transpose(1, 2)

    def forward(self, query, key, value, mask=None):
        q = self._split(self.w_q(query))
```

```
        k = self._split(self.w_k(key))
        v = self._split(self.w_v(value))
        out, self.attn = scaled_dot_product_attention(q, k, v, mask)
        B, _, L, _ = out.shape
        out = out.transpose(1, 2).contiguous().view(B, L, self.h * self.d_k)
        return self.dropout(self.w_o(out))


class FeedForward(nn.Module):
    """逐位置前馈网络: FFN(x) = W2 · ReLU(W1 x + b1) + b2"""

    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(d_ff, d_model), nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)


class EncoderLayer(nn.Module):
    """Post-LN（与原论文一致）：x = LN(x + Sublayer(x))"""

    def __init__(self, d_model, n_heads, d_ff, dropout):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, n_heads, dropout)
        self.ffn = FeedForward(d_model, d_ff, dropout)
        self.ln1, self.ln2 = nn.LayerNorm(d_model), nn.LayerNorm(d_model)

    def forward(self, x, src_mask):
        x = self.ln1(x + self.self_attn(x, x, x, src_mask))
        x = self.ln2(x + self.ffn(x))
        return x


class DecoderLayer(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, n_heads, dropout)   # 带因果掩码
        self.cross_attn = MultiHeadAttention(d_model, n_heads, dropout)  # 看编码器
        self.ffn = FeedForward(d_model, d_ff, dropout)
        self.ln1 = nn.LayerNorm(d_model)
        self.ln2 = nn.LayerNorm(d_model)
        self.ln3 = nn.LayerNorm(d_model)

    def forward(self, y, memory, tgt_mask, src_mask):
        y = self.ln1(y + self.self_attn(y, y, y, tgt_mask))
        y = self.ln2(y + self.cross_attn(y, memory, memory, src_mask))
        y = self.ln3(y + self.ffn(y))
        return y


class Transformer(nn.Module):
    def __init__(self, src_vocab, tgt_vocab, d_model=128, n_heads=4,
                 n_layers=2, d_ff=256, dropout=0.1):
        super().__init__()
        self.src_emb = TokenEmbedding(src_vocab, d_model)
        self.tgt_emb = TokenEmbedding(tgt_vocab, d_model)
        self.pos = PositionalEncoding(d_model, dropout=dropout)
```

```
        self.encoder = nn.ModuleList(
            [EncoderLayer(d_model, n_heads, d_ff, dropout) for _ in range(n_layers)])
        self.decoder = nn.ModuleList(
            [DecoderLayer(d_model, n_heads, d_ff, dropout) for _ in range(n_layers)])
        self.generator = nn.Linear(d_model, tgt_vocab)   # 输出投影到英文词表
        for p in self.parameters():
            if p.dim() > 1:
                nn.init.xavier_uniform_(p)

    # ---------- 掩码 ----------
    @staticmethod
    def make_src_mask(src):                    # (B,Ls) -> (B,1,1,Ls)
        return (src != PAD_IDX).unsqueeze(1).unsqueeze(2)

    @staticmethod
    def make_tgt_mask(tgt):                    # (B,Lt) -> (B,1,Lt,Lt)
        Lt = tgt.size(1)
        pad = (tgt != PAD_IDX).unsqueeze(1).unsqueeze(2)                  # (B,1,1,Lt)
        causal = torch.tril(torch.ones(Lt, Lt, dtype=torch.bool, device=tgt.device))
        return pad & causal                                               # 下三角

    # ---------- 前向 ----------
    def encode(self, src, src_mask):
        x = self.pos(self.src_emb(src))
        for layer in self.encoder:
            x = layer(x, src_mask)
        return x

    def decode(self, tgt, memory, tgt_mask, src_mask):
        y = self.pos(self.tgt_emb(tgt))
        for layer in self.decoder:
            y = layer(y, memory, tgt_mask, src_mask)
        return y

    def forward(self, src, tgt_in):
        src_mask = self.make_src_mask(src)
        tgt_mask = self.make_tgt_mask(tgt_in)
        memory = self.encode(src, src_mask)
        out = self.decode(tgt_in, memory, tgt_mask, src_mask)
        return self.generator(out)             # (B, Lt, |V_tgt|) 的 logits


# ============================================================
# 第 3 部分：训练
# ============================================================
def train(model, epochs=300, batch_size=8, lr=5e-4):
    opt = torch.optim.Adam(model.parameters(), lr=lr, betas=(0.9, 0.98), eps=1e-9)
    loss_fn = nn.CrossEntropyLoss(ignore_index=PAD_IDX, label_smoothing=0.1)
    model.train()
    for ep in range(1, epochs + 1):
        random.shuffle(PAIRS)
        total = 0.0
        for i in range(0, len(PAIRS), batch_size):
            src, tgt = make_batch(PAIRS[i:i + batch_size])
            tgt_in, tgt_out = tgt[:, :-1], tgt[:, 1:]   # Teacher forcing：右移一位
            logits = model(src, tgt_in)
            loss = loss_fn(logits.reshape(-1, logits.size(-1)), tgt_out.reshape(-1))
            opt.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
```

```
            total += loss.item()
        if ep % 50 == 0 or ep == 1:
            print(f"epoch {ep:4d} | loss {total / math.ceil(len(PAIRS) / batch_size):.4f}")


# ============================================================
# 第 4 部分：推理（贪心解码）
# ============================================================
@torch.no_grad()
def translate(model, zh_sentence, max_len=20, verbose=False):
    model.eval()
    src = torch.tensor([SRC_VOCAB.encode(zh_sentence)], device=DEVICE)
    src_mask = model.make_src_mask(src)
    memory = model.encode(src, src_mask)                    # 编码器只跑一次
    ys = torch.tensor([[TGT_VOCAB.stoi[BOS]]], device=DEVICE)
    for step in range(max_len):
        out = model.decode(ys, memory, model.make_tgt_mask(ys), src_mask)
        prob = torch.softmax(model.generator(out[:, -1]), dim=-1)
        p, next_id = prob.max(dim=-1)
        if verbose:
            print(f"  step {step + 1}: 已生成 {TGT_VOCAB.decode(ys[0].tolist())!r:32s}"
                  f" -> 预测 {TGT_VOCAB.itos[next_id.item()]!r} (p={p.item():.3f})")
        ys = torch.cat([ys, next_id.view(1, 1)], dim=1)
        if next_id.item() == TGT_VOCAB.stoi[EOS]:
            break
    return TGT_VOCAB.decode(ys[0].tolist())


def show_cross_attention(model, zh_sentence):
    """打印最后一层解码器的交叉注意力（多头平均），看每个英文词“对齐”到哪个中文词"""
    en = translate(model, zh_sentence)
    src = torch.tensor([SRC_VOCAB.encode(zh_sentence)], device=DEVICE)
    tgt = torch.tensor([TGT_VOCAB.encode(en)[:-1]], device=DEVICE)   # <bos> + 译文
    with torch.no_grad():
        model(src, tgt)
    attn = model.decoder[-1].cross_attn.attn[0].mean(0)              # (Lt, Ls)
    src_toks = [BOS] + zh_sentence.split() + [EOS]
    out_toks = en.split() + [EOS]
    print("\n交叉注意力（行=生成的英文词，列=中文源词）")
    print("          " + "".join(f"{t:>8s}" for t in src_toks))
    for i, tok in enumerate(out_toks):
        print(f"{tok:>10s}" + "".join(f"{w:8.2f}" for w in attn[i].tolist()))


if __name__ == "__main__":
    print(f"中文词表 {len(SRC_VOCAB)} 个 token，英文词表 {len(TGT_VOCAB)} 个 token")
    model = Transformer(len(SRC_VOCAB), len(TGT_VOCAB)).to(DEVICE)
    print(f"模型参数量: {sum(p.numel() for p in model.parameters()):,}")

    train(model)

    print("\n===== 翻译: 我要吃苹果 =====")
    result = translate(model, "我 要 吃 苹果", verbose=True)
    print(f"\n我要吃苹果  ->  {result}")

    show_cross_attention(model, "我 要 吃 苹果")

    print("\n===== 其他测试 =====")
    for s in ["她 要 吃 苹果", "我 不 要 喝 牛奶", "他 要 喝 水"]:
        print(f"{s.replace(' ', ''):8s} -> {translate(model, s)}")
```

第 12 章　运行结果与解读

12.1 完整运行日志

```
中文词表 17 个 token，英文词表 27 个 token
模型参数量：671,643
epoch   1 | loss 3.6087
epoch  50 | loss 0.6797
epoch 100 | loss 0.6526
epoch 150 | loss 0.6433
epoch 200 | loss 0.6400
epoch 250 | loss 0.6364
epoch 300 | loss 0.6353

===== 翻译：我要吃苹果 =====
  step 1: 已生成 ''                              -> 预测 'i' (p=0.931)
  step 2: 已生成 'i'                             -> 预测 'want' (p=0.915)
  step 3: 已生成 'i want'                        -> 预测 'to' (p=0.917)
  step 4: 已生成 'i want to'                     -> 预测 'eat' (p=0.916)
  step 5: 已生成 'i want to eat'                 -> 预测 'an' (p=0.925)
  step 6: 已生成 'i want to eat an'              -> 预测 'apple' (p=0.933)
  step 7: 已生成 'i want to eat an apple'        -> 预测 '<eos>' (p=0.924)

我要吃苹果  ->  i want to eat an apple

交叉注意力（行=生成的英文词，列=中文源词）
             <bos>       我       要       吃      苹果   <eos>
         i    0.03    0.96    0.00    0.00    0.00    0.00
      want    0.28    0.12    0.50    0.02    0.00    0.07
        to    0.12    0.06    0.53    0.03    0.08    0.19
       eat    0.12    0.02    0.01    0.58    0.02    0.25
        an    0.11    0.06    0.00    0.67    0.01    0.14
     apple    0.04    0.00    0.02    0.41    0.50    0.03
     <eos>    0.17    0.03    0.01    0.05    0.02    0.72

===== 其他测试 =====
她要吃苹果    -> she wants to eat an apple
我不要喝牛奶   -> i do not want to drink milk
他要喝水      -> he wants to drink water
```

在 CPU 上整个过程约 17 秒。

12.2 读懂训练曲线

- epoch 1 时 loss ≈ 3.6，接近 $\ln 27 \approx 3.3$——随机猜测的水平；
- epoch 50 时已降到 0.68，模型基本学会了；
- 之后缓慢逼近约 0.63 的下限（标签平滑带来的不可消除部分）。

12.3 读懂交叉注意力：模型自己学会了“对齐”

这是全书最值得细看的一张表。它展示解码器最后一层生成每个英文词时，交叉注意力把权重分配在哪些中文词上：

生成的英文	最关注的中文	权重	解读
i	我	**0.96**	几乎完全对齐到主语
want	要	**0.50**	对齐到情态动词
to	要	**0.53**	“to”是“want to”结构的一部分，所以也看“要”
eat	吃	**0.58**	对齐到动词
an	吃	0.67	冠词出现在动词之后、宾语之前，模型在“吃”处判断接下来是宾语
apple	苹果 / 吃	**0.50** / 0.41	对齐到宾语，同时保留对动词的关注
`<eos>`	`<eos>`	**0.72**	源句已翻译完，关注源端结束符

没有任何人告诉过模型“我=i、吃=eat”。训练数据里只有整句对整句。词级对齐完全是模型为了降低翻译损失**自己学出来的**。这就是注意力机制最迷人的地方：它既是计算工具，又在一定程度上可被解释。

> 注意：注意力权重只是一个**参考**，不等于严格的因果解释。在大模型中，单个注意力头的含义往往更复杂。

12.4 泛化能力

输入	输出	说明
她要吃苹果	she wants to eat an apple	训练集内的句子，正确使用第三人称 wants
我不要喝牛奶	i do not want to drink milk	**该句不在训练集中**，模型组合了“我不要喝水”与“喝牛奶”的知识
他要喝水	he wants to drink water	**同样不在训练集中**，组合了“他→he wants”与“喝水→drink water”

“我不要喝牛奶”说明模型不是在背句子，而是学到了“不要 → do not want”、“牛奶 → milk（无冠词）”这样可组合的规则。当然，22 句的语料只能支持非常有限的泛化——遇到“我要吃面条”这种含未登录词的句子，它就无能为力了（“面条”会变成 `<unk>`）。

第 13 章　从玩具到真实系统

本书模型与真实翻译系统在**架构上完全相同**，差别在规模与工程细节：

方面	本书	原论文 Transformer-base	现代大模型
语料	22 句	WMT14 英德 450 万句对	万亿级 token
分词	手工空格分词	BPE（约 37k 子词）	BPE / SentencePiece（10 万级词表）
d_model	128	512	数千
层数	2 + 2	6 + 6	数十至上百
头数	4	8	数十
参数量	67 万	6500 万	数百亿到万亿
归一化	Post-LN	Post-LN	Pre-LN / RMSNorm
位置编码	正弦	正弦	RoPE 等
解码	贪心	Beam search (k=4)	Beam / 采样

13.1 想把它做成真正能用的翻译器

1. **数据**：使用公开中英平行语料，如 WMT 中英数据集、OPUS 系列（News Commentary、UN Parallel Corpus 等）；
2. **分词**：用 `sentencepiece` 训练一个共享或独立的子词模型，彻底解决未登录词问题；
3. **规模**：d_model=512、6+6 层、warmup 学习率、GPU 训练；
4. **评估**：用 BLEU / chrF / COMET 等指标在测试集上评估；
5. **或者直接微调预训练模型**：如 Helsinki-NLP 的 `opus-mt-zh-en`（基于 Marian，同样是编码器-解码器 Transformer），几行代码即可获得可用的翻译质量。

13.2 Transformer 的三大家族

本书实现的是完整的**编码器-解码器**结构。后来的发展把它拆成了三支：

家族	结构	代表	擅长
仅编码器	只用左半边，双向注意力	BERT	理解：分类、抽取、检索
编码器-解码器	完整结构	原始 Transformer、T5、BART、Marian	序列转换：翻译、摘要
仅解码器	只用右半边（去掉交叉注意力），因果掩码	GPT 系列及今天绝大多数大语言模型	生成：对话、写作、代码

今天的大语言模型做翻译，用的就是“仅解码器”结构：把“把下面的中文翻译成英文：我要吃苹果”作为前缀，然后自回归地续写——和本书第 10 章的推理过程是同一个原理。

附录 A　张量形状速查表

记号：B = 批大小，Ls = 源长度，Lt = 目标长度，d = d_model = 128，h = 头数 = 4，d_k = d/h = 32，V = 英文词表大小 = 27。

位置	张量	形状
输入	`src`	(B, Ls)
输入	`tgt_in`	(B, Lt)
嵌入 + 位置编码后	`x`	(B, Ls, d)
线性变换后拆头	`q, k, v`	(B, h, L, d_k)
注意力分数	`scores`	(B, h, Lq, Lk)
源端掩码	`src_mask`	(B, 1, 1, Ls)
目标端掩码	`tgt_mask`	(B, 1, Lt, Lt)
多头拼接后	`out`	(B, L, d)
编码器输出	`memory`	(B, Ls, d)
解码器输出	`out`	(B, Lt, d)
最终输出	`logits`	(B, Lt, V)

“我要吃苹果”单句推理时：`src` 为 (1, 6)，`memory` 为 (1, 6, 128)。

附录 B　常见错误与排查

现象	可能原因	解决
输出全是同一个词	学习率太大，或忘了因果掩码导致训练时“抄答案”	检查 `make_tgt_mask`；降低学习率
loss 出现 `nan`	某一行被全部屏蔽，softmax 全是 $-\infty$	确保每行至少有一个可见位置（`<bos>` 永远可见）
训练 loss 很低但推理乱码	训练/推理的输入不一致（如推理时忘加 `<bos>`）	对照第 9.2 节和第 10.2 节检查
翻译不停止	训练数据没加 `<eos>`	检查 `Vocab.encode` 是否添加了 `<eos>`
形状报错 `view`	转置后张量不连续	在 `view` 前加 `.contiguous()`
推理结果每次不同	忘了 `model.eval()`，dropout 仍在工作	推理前调用 `model.eval()`

附录 C　练习题

1. **去掉位置编码**：注释掉 `self.pos(...)`，重新训练。“我要吃苹果”还能正确翻译吗？试试“苹果 要 吃 我”会得到什么，解释原因。
2. **去掉因果掩码**：把 `make_tgt_mask` 改成只返回 `pad`。训练 loss 会怎样变化？推理结果呢？为什么？
3. **观察多个头**：修改 `show_cross_attention`，不对头取平均，分别打印 4 个头的注意力。它们是否关注了不同的东西？
4. **实现 Beam Search**：把 `translate` 改写成 beam size = 3 的版本。
5. **改成 Pre-LN**：把 `EncoderLayer` 改为 `x = x + self.self_attn(self.ln1(x), ...)` 的形式，比较收敛速度。
6. **扩展语料**：加入“我要吃面条 → i want to eat noodles”，看模型能否正确处理新的“无冠词可数名词复数”情形。
7. **反向翻译**：交换 `PAIRS` 中中英文的位置，训练一个英译中模型。需要修改哪些地方？

全书完。

参考文献

- Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS 2017. arXiv:1706.03762
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR 2015. arXiv:1409.0473
- Rush, A. (2018). *The Annotated Transformer*. Harvard NLP.
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer Normalization*. arXiv:1607.06450
- Xiong, R., et al. (2020). *On Layer Normalization in the Transformer Architecture*. ICML 2020.